Group Project: Multi-Agent Fake News Detection System

A Multi-Agent workflow with Crew.ai

Learning Objectives

In this group project, you will design and implement a **Multi-Agent System (MAS)** for automated fake news detection. Working with the Kaggle Fake News dataset, you will build a pipeline of specialized AI agents — each with a distinct role — that collaborate to classify articles as real or fake, explain their reasoning, and produce a structured verdict.

To prepare for the task you can read two original articles about agent workflows and use of tools [9, 6].

The goals of this assignment are to:

- Understand the principles of **multi-agent coordination** and task decomposition.
- Apply **Large Language Models (LLMs)** in an agentic, tool-using context.
- Use the **CrewAI** framework to orchestrate agents with defined roles, goals, and backstories.
- Evaluate agent performance against a labeled ground-truth dataset.
- Critically analyze the limitations and failure modes of LLM-based classifiers.

1 How the final system will look like?

Creativity has no limits, but here you have an idea on how could be the look and feel of your system

2 Some Tools for classifying Real and Fake

I will provide you with a large dataset with fake and real news. This is a combination of individual datasets that are circulating in different Kaggle competitions.

At the same time you'll have available a huggingface Fake/real detector. You can use it as a tool for one of the agents. As it is a classifiers it may help the judge when at doubt, can be considered a new signal for the final decision.

However, test your final model with your texts. Try to find fake news and real ones and test your system

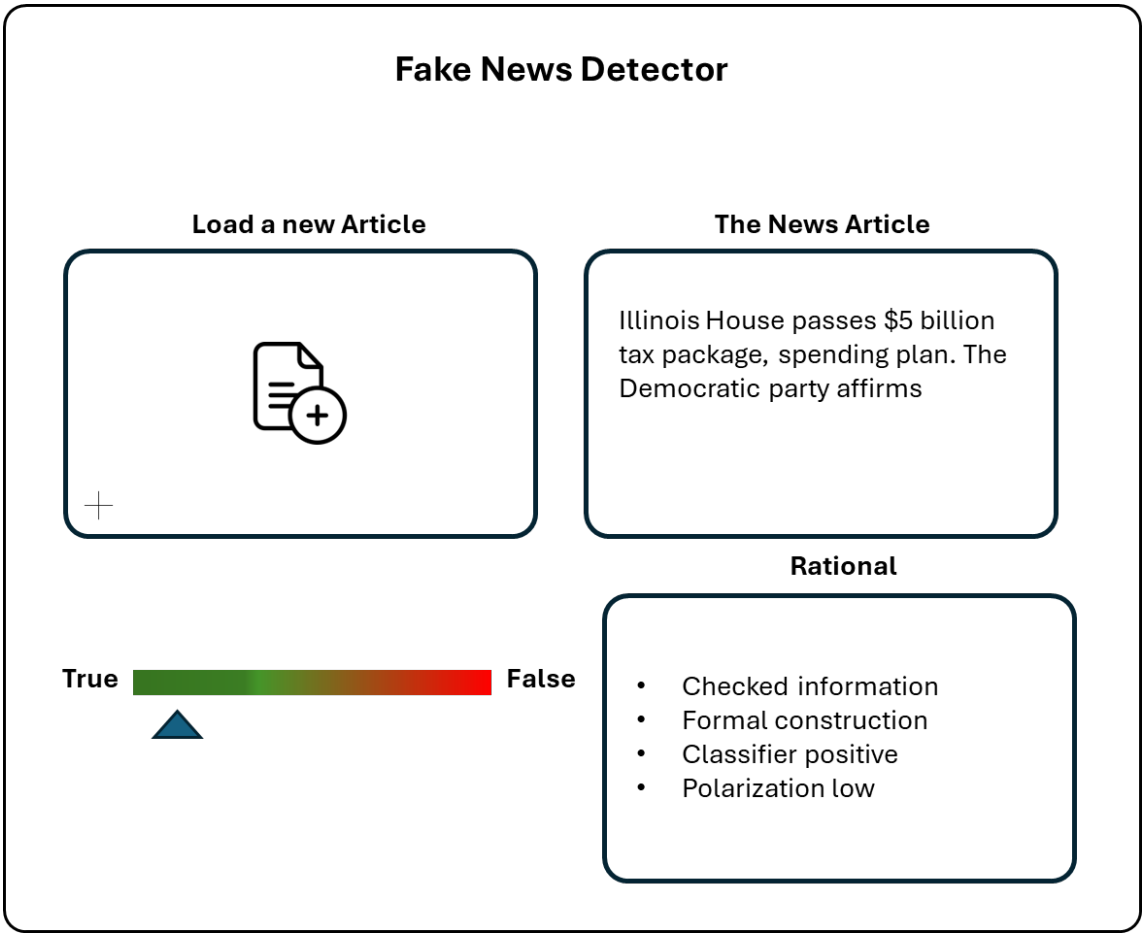

Figure 1: A Mock-up of the user interface.

2.1 Classification Model: RoBERTa for Fake News Detection

2.1.1 Base Architecture: RoBERTa

The classification model used in this work is based on RoBERTa (*Robustly Optimized BERT Pretraining Approach*), a transformer-based language model introduced by [3]. RoBERTa builds upon the BERT architecture [2], which consists of a stack of bidirectional self-attention encoder layers that produce rich contextual representations of input text. RoBERTa improves on BERT by addressing several suboptimal training decisions: it trains on significantly more data (over 160 GB of text), uses larger batch sizes, trains for longer, removes the Next Sentence Prediction (NSP) objective, and adopts dynamic masking patterns during pretraining. These modifications result in substantially stronger language representations, achieving state-of-the-art performance on benchmarks such as GLUE, SQuAD, and RACE.

The `roberta-base` variant used as the backbone of this model contains 12 transformer encoder layers, 768 hidden dimensions, 12 attention heads, and approximately 125 million parameters. Input text is tokenised using a Byte-Pair Encoding (BPE) tokeniser with a vocabulary of 50,265 tokens and a maximum sequence length of 512 tokens.

2.1.2 Fine-Tuning for Fake News Classification

The specific model employed in this work is `hamzab/roberta-fake-news-classification`,[1] a version of `roberta-base` fine-tuned for binary sequence classification. The fine-tuning task consists of predicting whether a given news article is *real* or *fake*. A linear classification head is added on top of the `[CLS]` token representation produced by the encoder, projecting from the hidden dimension to two output logits. A softmax function is applied to produce a probability distribution over the two classes.

The model was fine-tuned on the *Fake and Real News Dataset* [1], a widely used benchmark dataset available on Kaggle.[2] The dataset contains approximately 44,000 labelled news articles drawn from real-world sources, split evenly between fake and real news. Fake articles were collected from unreliable sources flagged by Politifact, while real articles were sourced from Reuters. According to the model card, the fine-tuned model achieves 100% accuracy on this dataset, suggesting strong alignment between the model's learned representations and the distributional patterns present in the training corpus.

2.1.3 Input Format

The model expects input structured according to the following template:

```
<title> TITLE HERE <content> CONTENT HERE <end>
```

where `<title>`, `<content>`, and `<end>` are special delimiter tokens that separate the headline from the article body. The combined input is truncated to 512 tokens to comply with the model's maximum sequence length. In this work, when the full article body exceeded the available token budget after the title, the body was progressively truncated to fit within the limit.

2.1.4 Inference

At inference time, the model returns a probability score for each class (*FAKE* and *REAL*). The predicted label is selected as the class with the highest probability:

$$\hat{y} = \arg\max_{c \in \{\text{FAKE},\text{REAL}\}} P(c \mid \mathbf{x}), \tag{1}$$

where $\mathbf{x}$ denotes the input token sequence. The model is accessed via the Hugging Face Inference API [8], which hosts the model and exposes it through a REST endpoint. Each article is submitted as a JSON payload and the API returns the class probabilities as a ranked list.

2.1.5 Limitations

Despite its strong in-distribution performance, several limitations of this model should be acknowledged. First, the training data is limited to English-language articles, primarily from a US political context, which may reduce generalisation to other domains or languages. Second, the reported 100% accuracy on the training dataset is indicative of overfitting to the specific stylistic patterns of the sources in that corpus,

[1] https://huggingface.co/hamzab/roberta-fake-news-classification
[2] https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset

and real-world performance on out-of-distribution articles may be considerably lower. Third, the model does not perform fact verification against external knowledge sources; classification decisions are based purely on surface-level linguistic patterns learned during fine-tuning.

3 Why CrewAI?

There are multiple frameworks available for building multi-agent systems: **LangGraph**, **AutoGen**, **LangChain Agents**, and **CrewAI**, among others. For this project, we recommend **CrewAI** for the following reasons:

- **Beginner-friendly abstraction:** Agents are defined declaratively with a `role`, `goal`, and `backstory`, removing the need to manage low-level prompt chains manually.
- **Built-in task delegation:** CrewAI handles the orchestration logic between agents automatically.
- **Tool integration:** Easy to attach web search, file reading, or custom Python tools to specific agents.
- **Process flexibility:** Supports both sequential pipelines and hierarchical (supervisor) architectures.

That said, you are free to implement the system in **plain Python with direct API calls** if you prefer more control. In that case, you are responsible for managing agent state, prompt construction, and output parsing yourself. Both approaches are valid and will be evaluated on the same criteria.

You can check information about **Crew.AI** in [4]

4 System Architecture

You must implement one of the two pipeline architectures described below. Both lead to the same four agents and the same final verdict — the difference is in **how and when** the Fact Checker and Bias Detector run relative to each other. Choose the architecture that best fits your implementation plan, and justify your choice in your final report.

4.1 Human Interface

You will generate a screen where to submit a file, then the system will process the file and respond with Fake - no fake. There will be a batch option to process sequentially a set of files and respond with a sequential file that will contain the fake - no fake information.

The recommendation is to use Streamlit. I would recommend that someone on your team specializes in the front-end or human interface.

4.2 Internal Agent Architecture

4.2.1 Option A — Sequential Pipeline

In the sequential architecture, each agent waits for the previous one to finish before starting. The Claim Extractor feeds the Fact Checker, which feeds the Bias Detector, which feeds the Judge. This is the simplest design to implement and debug.

When to choose this: You are new to multi-agent systems, or you want a clean, easy-to-trace execution log.

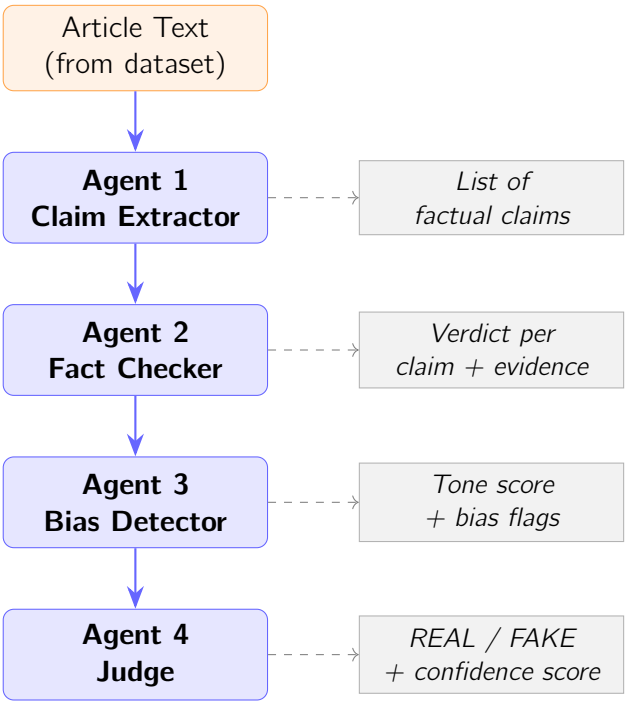

Figure 2: Option A: Sequential pipeline. Agents run one after another.

CrewAI configuration:

```
crew = Crew(
    agents=[claim_extractor, fact_checker, bias_detector, judge],
    tasks=[t1, t2, t3, t4],
    process=Process.sequential, # <-- key setting
    verbose=True
)
```

4.2.2 Option B — Parallel Pipeline

In the parallel architecture, the Fact Checker and the Bias Detector run **at the same time**, both receiving the article text and the list of claims simultaneously. Their outputs are then merged and passed to the Judge. This reduces total wall-clock time and models a more realistic team workflow where different analysts work concurrently.

When to choose this: You want a more challenging implementation, are comfortable with asynchronous Python, or want to explore the performance/cost tradeoff between architectures.

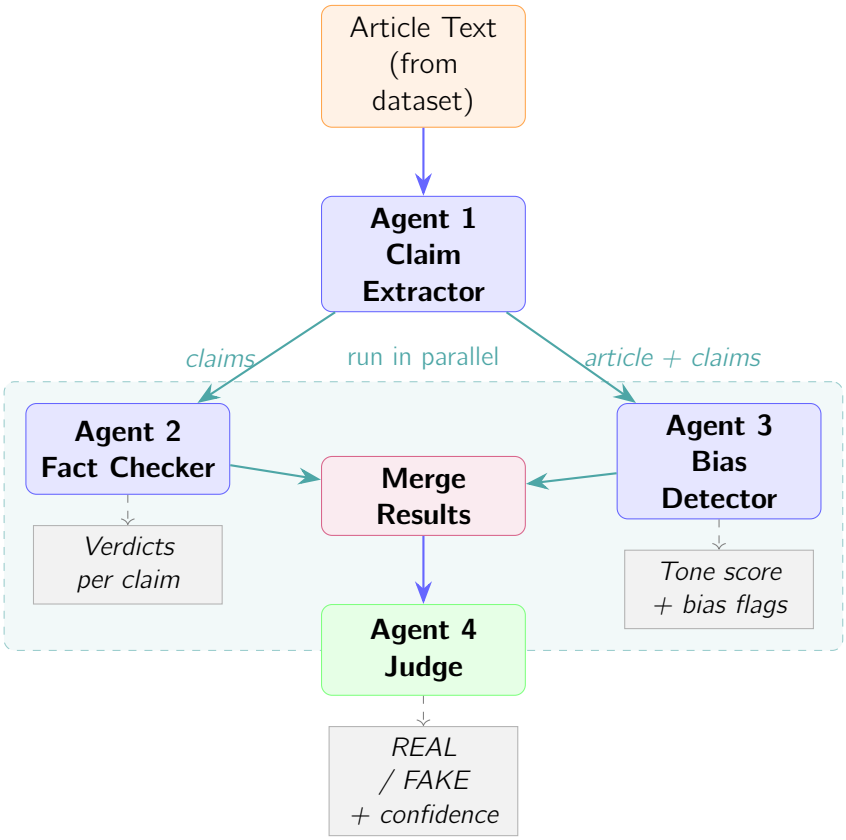

Figure 3: Option B: Parallel pipeline. Agents 2 and 3 run concurrently; results are merged before the Judge.

Implementation note — how to run agents in parallel:

CrewAI does not natively support task-level parallelism in its open-source version. There are two practical approaches:

Approach 1 — Python `concurrent.futures` (recommended):

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import ollama # or openai

def run_fact_checker(claims, article):
    # Call your fact_checker agent or LLM directly
    ...
    return fact_checker_output

def run_bias_detector(article):
    # Call your bias_detector agent or LLM directly
    ...
    return bias_detector_output

# Run both agents simultaneously
```

```
with ThreadPoolExecutor(max_workers=2) as executor:
    future_fc = executor.submit(run_fact_checker, claims, article)
    future_bd = executor.submit(run_bias_detector, article)

    fc_result = future_fc.result() # blocks until done
    bd_result = future_bd.result() # blocks until done

# Pass merged results to Judge
merged = {"fact_check": fc_result, "bias": bd_result}
judge_result = run_judge(merged)
```

Approach 2 — `asyncio` with async LLM calls:

```
import asyncio

async def run_parallel_agents(claims, article):
    # Fire both agents concurrently
    fc_task = asyncio.create_task(async_fact_checker(claims))
    bd_task = asyncio.create_task(async_bias_detector(article))

    # Await both
    fc_result, bd_result = await asyncio.gather(fc_task, bd_task)
    return fc_result, bd_result

fc_out, bd_out = asyncio.run(run_parallel_agents(claims, article_text))
```

4.3 Architecture Comparison

Property	Option A (Sequential)	Option B (Parallel)
Implementation effort	Low	Medium–High
Total execution time	Slower	Faster (Agents 2+3 overlap)
API cost	Same	Same
Debugging ease	Easy (linear trace)	Harder (concurrent logs)
Real-world analogy	One analyst at a time	Team working in parallel
Recommended for	Beginners	Intermediate students

Note: Regardless of which architecture you choose, the four agents, their roles, their output formats, and the evaluation methodology remain identical. Only the execution order of Agents 2 and 3 differs.

4.4 Agent 1 — Claim Extractor

Role: Read the article and extract a concise list of the key factual claims being made. Focus on verifiable statements (e.g., statistics, named entities, events, dates) and ignore opinions or rhetorical language.

Output format:

```
{
  "claims": [
    "The unemployment rate dropped to 3.4% in January 2024",
    "Senator X voted against the healthcare bill",
    "The study was conducted by Harvard University"
  ]
}
```

Hint: Limit the number of claims to 3–5 per article. Too many claims will make the fact-checking step slow and expensive.

4.5 Agent 2 — Fact Checker

Role: For each claim extracted by Agent 1, search for supporting or contradicting evidence and assign a verdict.

Options for tool use:

Note: use Tavily for search [7] engine. It is not necessary and I recommend not to include search capabilities at this time, however, if you'd like to do a more professional setup you can add search to the agents workflow.

- **Web search** (e.g., Tavily API, SerpAPI) — recommended for full implementation
- **LLM knowledge only** — acceptable for a simplified version (note its limitations)
- **Wikipedia API** — good balance of coverage and simplicity

Output format:

```
{
  "results": [
    {
      "claim": "The unemployment rate dropped to 3.4%",
      "verdict": "SUPPORTED",
      "confidence": 0.85,
      "evidence": "Bureau of Labor Statistics confirmed..."
    },
    ...
  ]
}
```

Possible verdicts: `SUPPORTED`, `CONTRADICTED`, `UNVERIFIABLE`.

4.6 Agent 3 — Bias Detector

Role: Analyze the language and tone of the article independently of its factual content. Identify emotional manipulation, loaded language, sensationalist headlines, one-sided framing, or missing context.

Output format:

```
{
  "tone": "sensationalist",
  "bias_score": 0.72, // 0 = neutral, 1 = highly biased
  "flags": [
    "Emotionally loaded headline",
    "No counter-arguments presented",
    "Unattributed quotes"
  ]
}
```

Hint: This agent does *not* need web search. It works purely on the text of the article.

4.7 Agent 4 — Judge

Role: Aggregate the outputs of all three previous agents and produce a final classification with a confidence score and a human-readable explanation. This agent acts as the decision-maker of the pipeline.

Output format:

```
{
  "label": "FAKE",
  "confidence": 0.88,
  "summary": "2 out of 3 claims were contradicted by available evidence.
              The article also uses highly sensationalist language
              and presents no counterarguments.",
  "agent_inputs_summary": {
      "claim_verdicts": ["CONTRADICTED", "UNVERIFIABLE", "CONTRADICTED"],
      "bias_score": 0.72,
      "tone": "sensationalist"
  }
}
```

4.7.1 Tools

For the Judge Agent You can use a tool to support the analysis. This is an AI tool developed to classify news in Fake/no-Fake. `https://huggingface.co/hamzab/roberta-fake-news-classification` This tool is a fine-tuned version of a BERT transformer. Is quite precise in political articles and I have prepared a Notebook that shows how it works. In the provided dataset obtains a 90% accuracy, which can be used by the judge to check the conclusions of the fact checkers. You can see the Notebook here `https://github.com/castorgit/AGENTS-course/blob/main/GROUP_PROJECT/Test_Fake_News_Detector_02.ipynb`

The problem I find with this classifier is that the data (you may find it in blackboard) is not of very good quality. The fake news are quite obvious I did some cleaning, but I think it is not a very good dataset of fake news, so that it may mislead in your workflow. However if you are interested, you can have a look.

It could be good to have an agent that acts as a judge offering a yes/no answer that you need to validate with the other agents.

5 Implementation Guide

5.1 Environment Setup

```
pip install crewai crewai-tools openai pandas scikit-learn
```

You will need an API key for your chosen LLM (OpenAI GPT-4o recommended, or any compatible model). Store your key in a `.env` file and load it with `python-dotenv`.

5.2 CrewAI Skeleton

Below is a minimal skeleton to get you started. You are expected to expand each component significantly.

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool

search_tool = SerperDevTool() # or use TavilySearchTool

# Define Agents
claim_extractor = Agent(
    role="Claim Extractor",
    goal="Extract verifiable factual claims from a news article",
    backstory="""You are an expert journalist and fact-checker.
                You are trained to identify the core factual
                assertions in any news article.""",
    verbose=True,
    allow_delegation=False
)

fact_checker = Agent(
    role="Fact Checker",
    goal="Verify each claim using web search and assign verdicts",
    backstory="""You are a senior researcher at a fact-checking
                organization. You cross-reference claims against
                reliable sources and never fabricate evidence.""",
    tools=[search_tool],
    verbose=True,
    allow_delegation=False
)

bias_detector = Agent(
    role="Bias Detector",
    goal="Analyze tone, framing, and language bias in news articles",
```

```
    backstory="""You are a linguistics professor specializing in
                media studies and computational rhetoric. You detect
                manipulation, emotional language, and framing bias.""",
    verbose=True,
    allow_delegation=False
)

judge = Agent(
    role="Judge",
    goal="Produce a final REAL/FAKE verdict based on all agent inputs",
    backstory="""You are the chief editor of a fact-checking platform.
                You synthesize evidence from your team and deliver
                clear, justified verdicts.""",
    verbose=True,
    allow_delegation=False
)

# Define Tasks
def build_tasks(article_text):
    t1 = Task(
        description=f"Extract the key factual claims from this article:\n\n{article_text}",
        agent=claim_extractor,
        expected_output="A JSON list of 3-5 verifiable factual claims."
    )
    t2 = Task(
        description="Verify each of the claims from the previous task using web search.",
        agent=fact_checker,
        expected_output="A JSON list of verdicts: SUPPORTED, CONTRADICTED, or UNVERIFIABLE.
            "
    )
    t3 = Task(
        description=f"Analyze the bias and tone of this article:\n\n{article_text}",
        agent=bias_detector,
        expected_output="A JSON object with bias_score, tone, and a list of bias flags."
    )
    t4 = Task(
        description="Based on all previous outputs, classify the article as REAL or FAKE.",
        agent=judge,
        expected_output="A JSON object with label, confidence, and a summary explanation."
    )
    return [t1, t2, t3, t4]

# Run the crew
article = "..." # Load from dataset
crew = Crew(
    agents=[claim_extractor, fact_checker, bias_detector, judge],
    tasks=build_tasks(article),
    process=Process.sequential,
```

```
    verbose=True
)
result = crew.kickoff()
```

5.3 Processing the Dataset

Since running the full agent pipeline on thousands of articles is expensive (in both time and API credits), you should **sample a subset** for evaluation. A sample of 50–100 articles is sufficient for this project.

```
import pandas as pd

df = pd.read_csv("train.csv").dropna(subset=["text"])
sample = df.sample(n=100, random_state=42)
```

Store all agent outputs in a results file for analysis. Do not re-run the pipeline unnecessarily.

6 LLM Backend: OpenAI API vs. Local Ollama

If your multi-agent with Ollama shows latency issues you can try with OPENAi **OpenAI API** [5]- (easiest, best quality) alternative to a **locally hosted model via Ollama** (free, private, no rate limits). Both options are compatible with CrewAI and require only a one-line change to swap between them.

You have used Ollama so far and you can keep using this setup, but if you see that your latency is a problem, you may like to have a setup with better latency. You can accomplish that using OpenAI.

6.1 Option 2 — Local Ollama (Free, Privacy-Preserving)

Thisis the same setup that you followed in previous assignments, you'll find here the instructions end to end

Step 1 — Install Ollama and pull a model.

```
# macOS
brew install ollama

# Linux
curl -fsSL https://ollama.com/install.sh | sh

# Pull a capable model (choose based on your hardware)
ollama pull llama3.2 # 3B params - fast, low RAM
ollama pull mistral # 7B params - good balance
ollama pull qwen2.5:7b # 7B params - strong reasoning
ollama pull llama3.1:8b # 8B params - recommended default

# Verify the model is available
ollama list
```

Step 2 — Start the Ollama server. Ollama must be running before your Python script starts. In a separate terminal:

```
ollama serve
# Server is now listening at http://localhost:11434
```

Step 3 — Configure CrewAI to use Ollama. CrewAI supports Ollama natively through its `LLM` class. The only change from the OpenAI setup is the model string — everything else (agents, tasks, crew) stays identical:

```
from crewai import Agent, LLM

# Switch from OpenAI to Ollama with one line
llm = LLM(
    model="ollama/llama3.1:8b", # prefix must be "ollama/"
    base_url="http://localhost:11434",
    temperature=0.2
)

claim_extractor = Agent(
    role="Claim Extractor",
    goal="Extract verifiable factual claims from a news article",
    backstory="You are an expert journalist and fact-checker.",
    llm=llm, # same argument, different backend
    verbose=True
)
```

6.2 Option 1 — OpenAI API (Recommended to Start)

The OpenAI API gives you access to all the OpenAI models with no local setup. It is the fastest way to get your pipeline running, but every request generates tokens and your data is sent to OpenAI's servers. The cost of tokens for this kind of exercise is very low, use it only if you set up a limit of cost in the OpenAI platform (you can set up limits as low as 5€)

If you feel like to give it a try this is the full set of steps to be taken

Step 1 — Create an API key. Go to `https://platform.openai.com/api-keys`, log in, and click *Create new secret key*. Copy the key immediately — it will not be shown again.

Step 2 — Store the key safely. Never hard-code your API key in a Python file. Store it in a `.env` file at the root of your project:

```
# .env (add this file to .gitignore immediately!)
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxx
```

Then load it in Python before making any API calls:

```python
from dotenv import load_dotenv
import os

load_dotenv() # reads .env and injects into environment
api_key = os.getenv("OPENAI_API_KEY")
```

Install the required packages:

```
pip install openai python-dotenv crewai
```

Step 3 — Configure CrewAI to use OpenAI. CrewAI reads `OPENAI_API_KEY` from the environment automatically. You only need to specify the model name when defining your agents:

```python
from crewai import Agent, LLM

llm = LLM(model="gpt-4o", temperature=0.2)

claim_extractor = Agent(
    role="Claim Extractor",
    goal="Extract verifiable factual claims from a news article",
    backstory="You are an expert journalist and fact-checker.",
    llm=llm,
    verbose=True
)
# Pass the same llm= argument to all other agents
```

Cost awareness. GPT-4o charges per token consumed. A single article processed through your 4-agent pipeline typically uses 3,000–8,000 tokens total. At current pricing (~$5 per million input tokens for GPT-4o), running 100 articles costs roughly $1.50–$4.00. This adds up quickly during development. Use `gpt-3.5-turbo` for early testing (~10× cheaper) and switch to `gpt-4o` only for final evaluation runs.

Model	Quality	Speed	Cost (approx.)
gpt-4o	Excellent	Medium	$$$
gpt-4o-mini	Very good	Fast	$
gpt-3.5-turbo	Good	Fast	$

Important — protect your key. Add `.env` to your `.gitignore` before your first `git commit`. A leaked key can result in hundreds of dollars of charges billed to your account within hours.

```
# .gitignore
.env
*.env
```

Step 4 — Make the backend switchable. A clean pattern is to control the backend via an environment variable, so you can switch with a single change in your `.env` file:

```python
import os
from dotenv import load_dotenv
from crewai import LLM

load_dotenv()

BACKEND = os.getenv("LLM_BACKEND", "openai") # default: openai

if BACKEND == "openai":
    llm = LLM(model="gpt-4o-mini", temperature=0.2)

elif BACKEND == "ollama":
    llm = LLM(
        model="ollama/llama3.1:8b",
        base_url="http://localhost:11434",
        temperature=0.2
    )
else:
    raise ValueError(f"Unknown backend: {BACKEND}")
```

Then in your `.env` file, simply change one line:

```
# To use OpenAI:
LLM_BACKEND=openai
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxx

# To use Ollama (comment out or remove OPENAI_API_KEY):
LLM_BACKEND=ollama
```

6.3 Comparison and Recommendations

Property	OpenAI API	Ollama (Local)
Setup effort	Low	Medium
Cost per 100 articles	$1.50–$4.00	Free
Model quality	Excellent (GPT-4o)	Good (7–8B models)
Internet required	Yes	No (after download)
Data privacy	Data sent to OpenAI	Stays on your machine
Rate limits	Yes	None
Response speed	Fast	Depends on hardware
Best for	Final evaluation runs	Development & testing

Recommended workflow for this project:

1. **Development phase:** Use Ollama with `llama3.1:8b` or `qwen2.5:7b`. Iterate freely on your prompts, agent backstories, and pipeline logic at zero cost.

2. **Evaluation phase:** Switch to `gpt-4o-mini` or `gpt-4o` for your final 100-article evaluation run to get the best possible classification performance for your report.
3. **Report:** Include a brief comparison of results between the two backends — this makes for an interesting analysis section and demonstrates understanding of the quality/cost tradeoff.

Hardware guide for Ollama model selection:

Your Machine	Recommended Model	Speed
Laptop, no GPU (8GB RAM)	`llama3.2:3b`	~5–10 tok/s
Laptop, no GPU (16GB RAM)	`llama3.1:8b`	~8–15 tok/s
Laptop with GPU (8GB VRAM)	`qwen2.5:7b`	~30–60 tok/s
Workstation (16GB+ VRAM)	`qwen2.5:14b`	~40–80 tok/s

7 Evaluation

Compare the Judge agent's predicted label against the ground-truth label from the dataset. Compute the following metrics:

- **Accuracy**: Overall percentage of correct predictions.
- **Precision, Recall, F1-score**: Per class (REAL and FAKE).
- **Confusion Matrix**: Visualize false positives and false negatives.

```
from sklearn.metrics import classification_report, confusion_matrix

y_true = [...] # ground truth labels from dataset
y_pred = [...] # labels predicted by Judge agent

print(classification_report(y_true, y_pred, target_names=["REAL", "FAKE"]))
```

Additionally, analyze the outputs qualitatively: look at cases where the system was wrong and try to understand why. This reflection is an important part of the final report.

8 Important Caveats and Known Limitations

Before drawing conclusions from your results, you should be aware of the following:

1. **LLMs can hallucinate evidence.** When the Fact Checker agent does not use real web search, it relies on the LLM's parametric knowledge, which may be outdated or fabricated. Always prefer a real search tool.
2. **The dataset has a cutoff date.** Some articles reference events that may now be publicly verified or debunked. Your agents may have an unfair advantage (or disadvantage) depending on the LLM's training cutoff.

3. **Bias detection is subjective.** Scores from the Bias Detector are not ground truth. They reflect the LLM's own tendencies and training data.
4. **API cost.** Processing 100 articles through a 4-agent pipeline with web search can cost several dollars in API credits. Plan accordingly and cache your results.

These limitations are not problems to hide — they are things to **discuss explicitly** in your final report.

9 Deliverables and Submission

You must submit the following:

- **Notebook 1 — Pipeline Implementation** (`.ipynb`): Full CrewAI pipeline with all four agents. Must be reproducible with a fresh API key.
- **Notebook 2 — Evaluation & Analysis** (`.ipynb`): Metrics, confusion matrix, error analysis, and qualitative discussion.
- **Results file** (`results.csv`): A CSV file containing article IDs, predicted labels, confidence scores, and agent outputs.
- **Presentation slides** (PowerPoint, Canva, or PDF): 10–15 minutes in-class demo and discussion.

You may add additional files (e.g., a `README.md` with setup instructions, a `.env.example`) if helpful.

Tip: Your presentation should include at least one live (or recorded) demo showing the pipeline processing a specific article, with all agent outputs visible.

References

[1] Hadeer Ahmed, Issa Traore, and Sherif Saad. "Detecting Opinion Spams and Fake News Using Text Classification". In: *Security and Privacy* 1.1 (2018), e9.

[2] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2019, pp. 4171–4186.

[3] Yinhan Liu et al. "RoBERTa: A Robustly Optimized BERT Pretraining Approach". In: *arXiv preprint arXiv:1907.11692* (2019). URL: `https://arxiv.org/abs/1907.11692`.

[4] Joao Moura. *CrewAI: Orchestrating Role-Playing, Collaborative AI Agents*. Version 0.30.0. Accessed: April 2026. 2024. URL: `https://github.com/joaomoura/crewai`.

[5] OpenAI. *OpenAI API Reference Documentation*. Accessed: April 2026. 2024. URL: `https://platform.openai.com/docs`.

[6] Timo Schick et al. "Toolformer: Language Models Can Teach Themselves to Use Tools". In: *arXiv preprint arXiv:2302.04761* (2023).

[7] Tavily AI. *Tavily Search API: Optimized for LLM Agents*. Accessed: April 2026. 2024. URL: `https://tavily.com`.

[8] Thomas Wolf et al. "Transformers: State-of-the-Art Natural Language Processing". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. Association for Computational Linguistics, 2020, pp. 38–45.

[9] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *arXiv preprint arXiv:2210.03629* (2022).